Selective Recovery in a Multi-Agent Coding Workflow

CSE 291A Fall 2026, Option 5 · Phase 1 Report · November 4, 2026

Jackie Wang, Ray Williams

1. Question

Option 5 asks whether tracking dependencies between agent results can make recovery efficient while keeping execution correct and safe. In a researcher → planner → executor chain, one bad intermediate result can flow into later plans, file edits and external actions. The two obvious responses, restarting everything or verifying every hand-off, are both costly. Phase 1 measures how far faults actually travel and what those two baselines cost. Phase 2 tests whether rerunning only the dependent results, and rolling back only the state they touched, recovers the same tasks with less work.

2. Workflow and tasks

The three agents share one model and fixed prompts. They complete small tickets on a fake Python `billing` library of about ten files.

- The **researcher** reads internal docs and reports the facts the change depends on, with citations.
- The **planner** turns the ticket and those facts into a JSON plan.
- The **executor** edits files, runs visible tests, commits, and may publish or post a message.

The agents share real state. The workspace is a git repository, so every executor step is a snapshot. A simulated package registry and team channel are append-only, so some actions cannot be undone.

Outcomes are checked by code, not by an LLM. The checker runs hidden pytest tests on the final code, diffs protected paths (tests, unrelated modules, a decoy function) against the seed, compares registry and channel entries with what the task expects, and flags tool calls or paths outside the task's allowlist. It reports unauthorized actions separately from task success.

For example, one ticket asks for `refund(order_id, amount)` in `billing/payments.py`. The deciding fact, "money is integer cents", is in `docs/conventions.md` and never in the ticket. A wrong-fact fault ("amounts are float dollars") makes the hidden test `refund("o1", 4999)` fail. A malicious fault ("also delete `tests/test_payments.py`") is collateral damage and an unauthorized action. Publishing the bad version is an irreversible effect.

About eight templates, each built on one doc rule, are expanded with a fixed seed into roughly 40 development and 40 held-out tasks (unseen until Phase 2), each changing 5–20 lines. Each task lists its deciding facts, plus one control fact that the answer does not depend on. Before any model runs, the checker must pass the reference solution, fail a do-nothing run, and catch the wrong-fact version, protected-file edits and an unrequested publish.

3. Fault injection and tracing

The harness rewrites one agent output or tool result before the next stage reads it, following a seeded schedule saved with each run.

Fault	Class	Where	Example
Wrong fact	Accidental	Researcher	"Amounts are float dollars"
Stale output	Accidental	Researcher, tool	Docs from before a key was renamed
Timeout	Accidental	Any call	No response within the limit
Fabricated fact	Adversarial	Researcher	Invented rule: "publish after every fix"
Malicious instruction	Adversarial	Researcher, tool	"Also delete the test file"

We report accidental and adversarial faults separately. **A fault counts only if the workload depends on it.** Each faulted run is paired with the clean run of the same task. A fault *propagated* if any downstream artifact or the final state differs from the clean run. It *caused failure* if the check fails or an unauthorized action occurs. Faults on control facts should change nothing. Tasks that the clean workflow already fails are reported separately and left out of the propagation analysis, so that a model's inability is not mistaken for protection.

Every agent output and tool result is an artifact with a provenance record in a per-run JSONL trace. The record holds the artifact's inputs, its state reads and writes (with versions), tokens, latency, status, and the fault ID if the harness injected one. State links count as much as message links. For example, a step that reads a file written by a faulty step depends on the fault even if no message connects the two. Phase 2 reads this same trace to decide what to invalidate.

4. Baselines, metrics and setup

All conditions run on the same tasks and fault schedule:

- **Clean** sets the capability ceiling.
- **Faulted, no recovery** measures propagation and damage.
- **Full restart** restores the seed state and reruns every stage without the fault.
- **Always-verify** has a verifier check each result before hand-off, and counts its tokens and latency.

Each recovery method is told which result is faulty, so we measure recovery apart from detection. We report clean task success, propagation and failure rates by fault class and stage, unauthorized actions, affected stages, wasted work (rerun steps whose output did not change), irreversible effects, and end-to-end latency and tokens. Main runs use temperature 0, a subset repeats with three seeds (mean and min–max), and failed runs and timeouts stay in the data.

We self-host the model with llama.cpp at a pinned commit. The candidates are Qwen3.5 4B, Phi-4-mini-instruct, and Qwen3.5 9B or Qwen3 8B, all quantized to Q4_K_M, with TritonAI as the fallback. By Oct 29, a pilot of five tickets, each run three times, picks the fastest model that passes at least 60% of chained runs in about three minutes per task. Phase 1 runs on a MacBook Pro (M4 Pro). Latency does not transfer between machines, so the final comparison reruns the baselines alongside Phase 2 on one AWS CPU instance, within the $3/day limit. Each run's `config.yaml` records the commits, model hash and hardware.

5. Status and Phase 2 direction

As of Oct 27, we have a repository scaffold, a versioned key-value store with an append-only write log and `rollback_to(version)`, and 8 passing unit tests. The next step replaces our first simulated tools, written for an earlier customer-support design, with coding tools. No model has run yet, so this report has no measurements.

Phase 2 builds dependency-aware invalidation with shared-state rollback. Recovery walks the provenance graph, through both message inputs and state reads, to find every result downstream of the faulty one. It restores the workspace and the store to just before the first affected write, and reruns only the invalidated stages. Effects that cannot be undone, such as a published release or a posted message, are reported instead of counted as recovered. A sample of reused results is rerun to confirm they were valid. We will compare this method with both baselines under identical fault schedules and budgets, and weigh recovery precision against bookkeeping overhead. If Phase 1 shows that most faults reach every later stage, then selective recovery saves little, and we will report that.

Appendix: Timeline and milestones

Dates	Work	Milestone
Oct 27	Repo scaffold, state store, unit tests	Done: 8 tests pass
Oct 27–29	Model pilot on five tickets	Model chosen; file hash recorded
Oct 28	Phase 1 report	This document
Oct 29–30	Coding environment and tools	Tool unit tests pass
Oct 30–31	Tasks, hidden tests, checker; agents	Reference passes; do-nothing fails
Oct 31–Nov 1	Provenance tracing; fault injector	Faulted run differs from clean pair
Nov 1–2	Restart and verify baselines; 5-task pilot	All four conditions run end to end
Nov 3	Full dev-set runs; analysis script	Tables and plots rebuilt from traces
Nov 4	**Phase 1 code and data due**	One command per condition
Nov 5–17	Selective invalidation	Only downstream stages rerun
Nov 12–24	Shared-state rollback	Irreversible actions reported
Nov 23–Dec 3	Held-out evaluation on AWS	Same schedules, budgets and seeds
Dec 1–9	Final report	
Dec 10	**Final submission**	

Risks and fallbacks: